

LC 88 - Merge Sorted Array

© Authored by Arpit Raj, LNMIIT Jaipur | July 15, 2026

TCS

Wipro

Infosys

Epam

HCL

Problem Context



Goal: merge them and return sorted $\rightarrow [1, 2, 2, 3, 5, 6]$

Note: extra 0 given for in place processing

Initial Thoughts (Naive Approach)

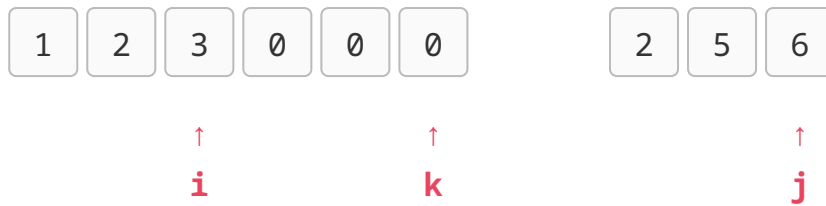
So simply keep adding from mth index upto (m+n) index and then sort

Can this be done without sorting?

If we use 2 pointer from first position then it can lead to overwriting.

Eg 2 from B is less than 3, so it will overwrite.

| Correct Approach -> 3 Pointer



1. compare (i and j) whoever is bigger place at position k and then $k--$

2. let's say j was bigger then

`nums[k] = nums[j]`

`j--;`

`k--;`

| Complexity Analysis

TC = $O(m+n)$

SC = $O(1)$

C++ Implementation

```
class Solution {
public:
    void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
        int i = m - 1;
        int j = n - 1;
        int k = n + m - 1;

        while(i >= 0 && j >= 0){
            if(nums1[i] > nums2[j]){
                nums1[k] = nums1[i];
                i--;
                k--;
            }
            else{
                nums1[k] = nums2[j];
                j--;
                k--;
            }
        }

        while(j >= 0){    //remaining elements from nums2
            nums1[k] = nums2[j];
            j--;
            k--;
        }
    }
};
```